Lab Assignment 9.1

1. Consider the following Python function

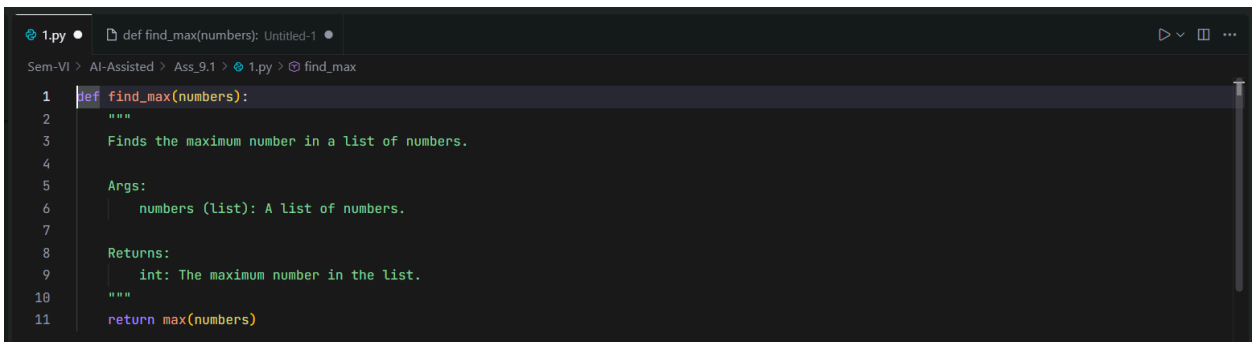
Consider the following Python function:

```python
def find_max(numbers):
    return max(numbers)
```

Task:

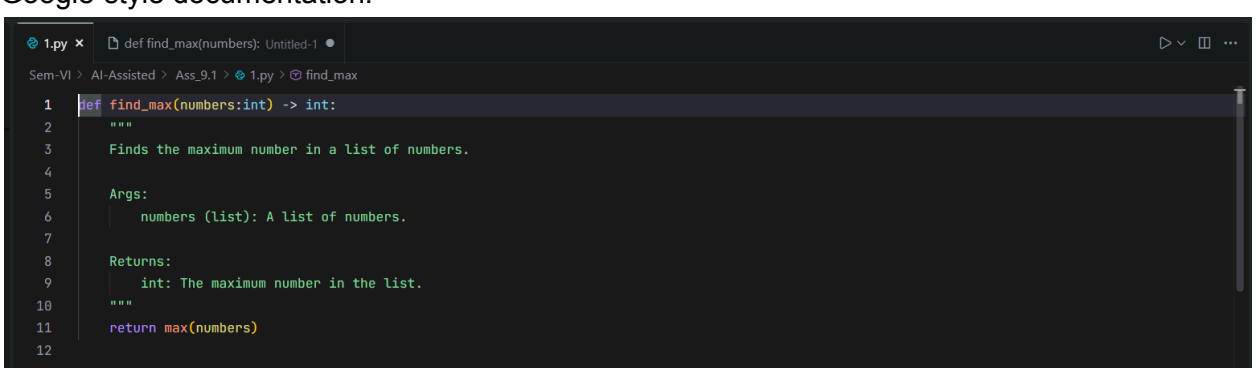
- Write documentation for the function in all three formats:
  - Docstring
  - Inline comments
  - Google-style documentation
- Critically compare the three approaches. Discuss the advantages, disadvantages, and suitable use cases of each style.
- Recommend which documentation style is most effective for a mathematical utilities library and justify your answer.

Docstring:



Inline Comments:

Google-style documentation:



2. Consider the following Python function:

```python
def login(user, password, credentials):
    return credentials.get(user) == password
```

Task:

- Write documentation in all three formats.
- Critically compare the approaches.
- Recommend which style would be most helpful for new developers onboarding a project, and justify your choice.

Docstring:

```python
def login(user, password, credentials):
    """
    Logs in a user with the given credentials.

    Args:
        user (str): The username of the user.
        password (str): The password of the user.
        credentials (dict): A dictionary of valid credentials.

    Returns:
        bool: True if the login is successful, False otherwise.
    """
    if user in credentials and credentials[user] == password:
        return True
    else:
        return False
```

Inline:

```python
def login(user, password, credentials):
    # Logs in a user with the given credentials.
    # Checks if the user exists in the credentials dictionary and if the password matches.
    if user in credentials and credentials[user] == password:
        return True # Valid password return True
    else:
        return False # Invalid password return False
```

Google-style documentation:

```python
def login(user: str, password: str, credentials: dict) -> bool:
    """
    Logs in a user with the given credentials.

    Args:
        user (str): The username of the user.
        password (str): The password of the user.
        credentials (dict): A dictionary of valid credentials.

    Returns:
        bool: True if the login is successful, False otherwise.
    """
    if user in credentials and credentials[user] == password:
        return True
    else:
        return False
```

3. Calculator (Automatic Documentation Generation)

Task:

Design a Python module named calculator.py and demonstrate automatic documentation generation.

Instructions:

- Create a Python module calculator.py that includes the following functions, each written with appropriate docstrings:
  - add(a, b) – returns the sum of two numbers
  - subtract(a, b) – returns the difference of two numbers
  - multiply(a, b) – returns the product of two numbers
  - divide(a, b) – returns the quotient of two numbers
- Display the module documentation in the terminal using Python's documentation tools.
- Generate and export the module documentation in HTML format using the pydoc utility, and open the generated HTML file in a web browser to verify the output.

1.py 2.py calculator.py ×

Sem-VI > AI-Assisted > Ass_9.1 > calculator.py > subtract

```python
def add(a: float, b: float) -> float:
    """
    Add two numbers.

    Args:
        a (float): The first number.
        b (float): The second number.

    Returns:
        float: The sum of a and b.
    """
    return a + b

def subtract(a: float, b: float) -> float:
    """
    Subtract two numbers.

    Args:
        a (float): The first number.
        b (float): The second number.

    Returns:
        float: The difference between a and b.
    """
    return a - b

```

1.py × 2.py calculator.py ×

Sem-VI > AI-Assisted > Ass_9.1 > calculator.py > subtract

```python

def multiply(a: float, b: float) -> float:
    """
    Multiply two numbers.

    Args:
        a (float): The first number.
        b (float): The second number.

    Returns:
        float: The product of a and b.
    """
    return a * b

def divide(a: float, b: float) -> float:
    """
    Divide two numbers.

    Args:
        a (float): The first number.
        b (float): The second number.

    Returns:
        float: The quotient of a and b.
    """
    return a / b

```

Terminal:

```
Problems   Output   Debug Console   Terminal                                   python
PS C:\Users\rhars\Documents\.Dev\Uni> python -m pydoc Sem-VI\AI-Assisted\Ass_9.1\calculator.py
Help on module calculator:

NAME
    calculator

FUNCTIONS
    add(a: float, b: float) -> float
        Add two numbers.

        Args:
            a (float): The first number.
            b (float): The second number.

        Returns:
            float: The sum of a and b.

    divide(a: float, b: float) -> float
        Divide two numbers.

        Args:
            a (float): The first number.
            b (float): The second number.

        Returns:
            float: The quotient of a and b.

    multiply(a: float, b: float) -> float
        Multiply two numbers.

        Args:
            a (float): The first number.
            b (float): The second number.

        Returns:
            float: The product of a and b.
```

```
Problems   Output   Debug Console   Terminal                                   powershell

    divide(a: float, b: float) -> float
        Divide two numbers.

        Args:
            a (float): The first number.
            b (float): The second number.

        Returns:
            float: The quotient of a and b.

    multiply(a: float, b: float) -> float
        Multiply two numbers.

        Args:
            a (float): The first number.
            b (float): The second number.

        Returns:
            float: The product of a and b.

    subtract(a: float, b: float) -> float
        Subtract two numbers.

        Args:
            a (float): The first number.
            b (float): The second number.

        Returns:
            float: The difference between a and b.

FILE
    c:\users\rhars\documents\.dev\uni\sem-vi\ai-assisted\ass_9.1\calculator.py
```

Generating Python HTML Doc:

```
Problems   Output   Debug Console   Terminal                                   powershell
PS C:\Users\rhars\Documents\.Dev\Uni> python -m pydoc -w Sem-VI\AI-Assisted\Ass_9.1\calculator.py
wrote calculator.html
PS C:\Users\rhars\Documents\.Dev\Uni>
```

index
calculator c:\users\rhars\documents\.dev\uni\sem-vi\ai-assisted\ass_9.1\calculator.py

Functions

add(a: float, b: float) -> float
Add two numbers.

Args:
a (float): The first number.
b (float): The second number.

Returns:
float: The sum of a and b.

divide(a: float, b: float) -> float
Divide two numbers.

Args:
a (float): The first number.
b (float): The second number.

Returns:
float: The quotient of a and b.

multiply(a: float, b: float) -> float
Multiply two numbers.

Args:
a (float): The first number.
b (float): The second number.

Returns:
float: The product of a and b.

subtract(a: float, b: float) -> float
Subtract two numbers.

Args:
a (float): The first number.
b (float): The second number.

Returns:
float: The difference between a and b.

4. Conversion Utilities Module

Task:

- Write a module named conversion.py with functions:
  - decimal_to_binary(n)
  - binary_to_decimal(b)
  - decimal_to_hexadecimal(n)
- Use Copilot for auto-generating docstrings.
- Generate documentation in the terminal.
- Export the documentation in HTML format and open it in a browser.

1.py × 2.py calculator.py conversion.py ...\Ass_9.1 × conversion.py .\

Sem-VI > AI-Assisted > Ass_9.1 > conversion.py > decimal_to_hexadecimal

```python
def decimal_to_binary(n:int) -> str:
    """
    Convert a decimal number to binary.

    Args:
        n (int): The decimal number to convert.

    Returns:
        str: The binary representation of n.
    """
    return bin(n)[2:]

def binary_to_decimal(b:int) -> int:
    """
    Convert a binary number to decimal.

    Args:
        b (int): The binary number to convert.

    Returns:
        int: The decimal representation of b.
    """
    return int(b, 2)

def decimal_to_hexadecimal(n:int) -> str:
    """
    Convert a decimal number to hexadecimal.

    Args:
        n (int): The decimal number to convert.

    Returns:
        str: The hexadecimal representation of n.
    """
    return hex(n)[2:]
```

Documentation in Terminal:

```
Problems   Output   Debug Console   Terminal                                   powershell
PS C:\Users\rhars\Documents\.Dev\Uni> python -m pydoc Sem-VI\AI-Assisted\Ass_9.1\conversion.py
Help on module conversion:

NAME
    conversion

FUNCTIONS
    binary_to_decimal(b: int) -> int
        Convert a binary number to decimal.

        Args:
            b (int): The binary number to convert.

        Returns:
            int: The decimal representation of b.

    decimal_to_binary(n: int) -> str
        Convert a decimal number to binary.

        Args:
            n (int): The decimal number to convert.

        Returns:
            str: The binary representation of n.

    decimal_to_hexadecimal(n: int) -> str
        Convert a decimal number to hexadecimal.

        Args:
            n (int): The decimal number to convert.

        Returns:
            str: The hexadecimal representation of n.

FILE
    c:\users\rhars\documents\.dev\uni\sem-vi\ai-assisted\ass_9.1\conversion.py

PS C:\Users\rhars\Documents\.Dev\Uni>
```

Exporting in HTML:

```
Problems   Output   Debug Console   Terminal
PS C:\Users\rhars\Documents\.Dev\Uni> python -m pydoc -w Sem-VI\AI-Assisted\Ass_9.1\conversion.py
wrote conversion.html
PS C:\Users\rhars\Documents\.Dev\Uni>
```

Displaying the HTML:

Python: module testt | Pydoc: Index of Modules | Python: module calculator | Python: module conversion

File C:/Users/rhars/Documents/.Dev/Uni/conversion.html

Import favorites | Foundations of LLM | C/C++ for Visual S... | Using GCC with Mi... | Close Your Accoun... | Other favorites

index
conversion c:\users\rhars\documents\.dev\uni\sem-vi\ai-assisted\ass_9.1\conversion.py

Functions

binary_to_decimal(b: int) -> int
Convert a binary number to decimal.

Args:
b (int): The binary number to convert.

Returns:
int: The decimal representation of b.

decimal_to_binary(n: int) -> str
Convert a decimal number to binary.

Args:
n (int): The decimal number to convert.

Returns:
str: The binary representation of n.

decimal_to_hexadecimal(n: int) -> str
Convert a decimal number to hexadecimal.

Args:
n (int): The decimal number to convert.

Returns:
str: The hexadecimal representation of n.

5. Course Management Module

Task:

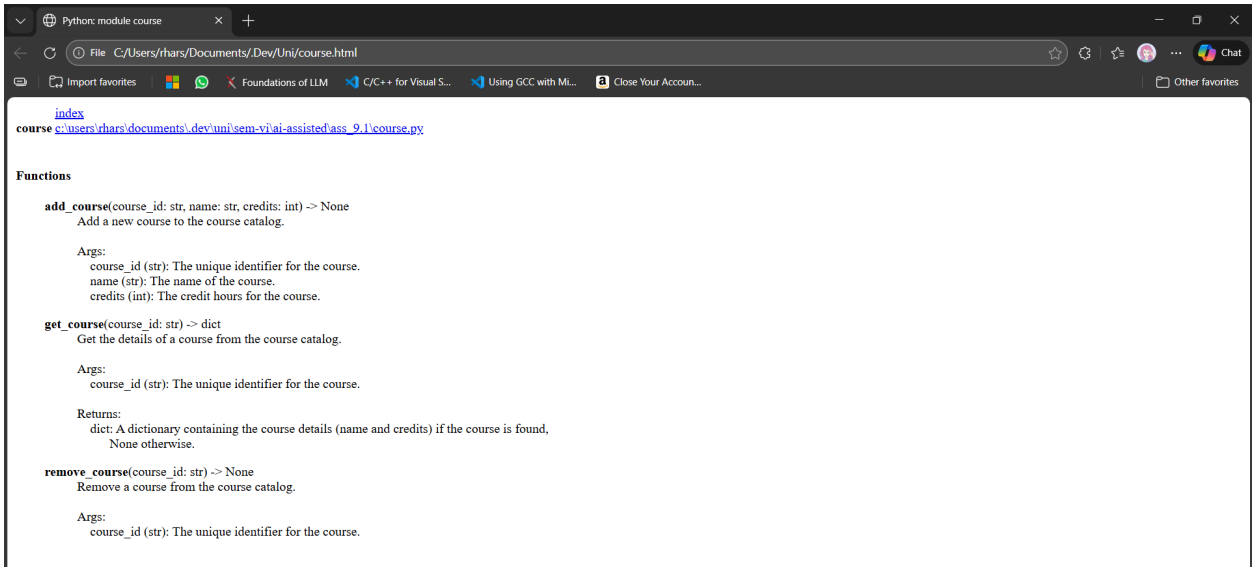
- Create a module course.py with functions:
  - add_course(course_id, name, credits)
  - remove_course(course_id)
  - get_course(course_id)
- Add docstrings with Copilot.
- Generate documentation in the terminal.
- Export the documentation in HTML format and open it in a browser.

```python
def add_course(course_id: str, name: str, credits: int) -> None:
    """
    Add a new course to the course catalog.

    Args:
        course_id (str): The unique identifier for the course.
        name (str): The name of the course.
        credits (int): The credit hours for the course.
    """
    # Add the course to the catalog
    course_catalog[course_id] = {'name': name, 'credits': credits}
    print(f"Course {course_id} added successfully.")


def remove_course(course_id: str) -> None:
    """
    Remove a course from the course catalog.

    Args:
        course_id (str): The unique identifier for the course.
    """
    # Remove the course from the catalog
    if course_id in course_catalog:
        del course_catalog[course_id]
        print(f"Course {course_id} removed successfully.")
    else:
        print(f"Course {course_id} not found in the catalog.")

def get_course(course_id: str) -> dict:
    """
    Get the details of a course from the course catalog.

    Args:
        course_id (str): The unique identifier for the course.

    Returns:
        dict: A dictionary containing the course details (name and credits) if the course is found,
              None otherwise.
    """
    # Get the course details from the catalog
    if course_id in course_catalog:
        return course_catalog[course_id]
    else:
        print(f"Course {course_id} not found in the catalog.")
        return None
```

Documentation in Terminal:

```
PS C:\Users\rhars\Documents\.Dev\Uni> python -m pydoc Sem-VI\AI-Assisted\Ass_9.1\course.py
Help on module course:

NAME
    course

FUNCTIONS
    add_course(course_id: str, name: str, credits: int) -> None
        Add a new course to the course catalog.

        Args:
            course_id (str): The unique identifier for the course.
            name (str): The name of the course.
            credits (int): The credit hours for the course.

    get_course(course_id: str) -> dict
        Get the details of a course from the course catalog.

        Args:
            course_id (str): The unique identifier for the course.

        Returns:
            dict: A dictionary containing the course details (name and credits) if the course is found,
                  None otherwise.

    remove_course(course_id: str) -> None
        Remove a course from the course catalog.

        Args:
            course_id (str): The unique identifier for the course.

FILE
    c:\users\rhars\documents\.dev\uni\sem-vi\ai-assisted\ass_9.1\course.py

PS C:\Users\rhars\Documents\.Dev\Uni>
```

HTML Documentation:



Python: module course

File C:/Users/rhars/Documents/.Dev/Uni/course.html

index

course c:\users\rhars\documents\.dev\uni\sem-vi\ai-assisted\ass_9.1\course.py

Functions

add_course(course_id: str, name: str, credits: int) -> None
Add a new course to the course catalog.

Args:
course_id (str): The unique identifier for the course.
name (str): The name of the course.
credits (int): The credit hours for the course.

get_course(course_id: str) -> dict
Get the details of a course from the course catalog.

Args:
course_id (str): The unique identifier for the course.

Returns:
dict: A dictionary containing the course details (name and credits) if the course is found,
None otherwise.

remove_course(course_id: str) -> None
Remove a course from the course catalog.

Args:
course_id (str): The unique identifier for the course.